```
    expect(post: "/projects").to route_to(
      controller: "projects", action: "create")
    expect(get: "/projects/new").to route_to(
      controller: "projects", action: "new")
    expect(get: "/projects/1").to route_to(
      controller: "projects", action: "show", id: "1")
    expect(get: "/projects/1/edit").to route_to(
      controller: "projects", action: "edit", id: "1")
    expect(patch: "/projects/1").to route_to(
      controller: "projects", action: "update", id: "1")
    expect(delete: "/projects/1").to route_to(
      controller: "projects", action: "destroy", id: "1")
  end
end
```

All of these are using the same form. The argument to expect is a key/value pair where the key is the HTTP verb and the value is the string form of the route. The argument to route_to is a set of key/value pairs where the keys are the parts of the calculated route (including controller, action, and what have you) and the values are, well, the values.

The route_to matcher tests the routes in both directions. It checks that when you send the path through the routing engine, you get the controller, action, and other variables specified. It also checks that the set of controller, action, and other variables sent through the router results in the path string (which is why you might need to specify query-string elements). It's not clear to me why a route might pass in one direction and not the other.

RSpec also provides a be_routable method, which is designed to be used in the negative to show that a specific path—say, the Rails default—is not recognized:

```
expect(get: "/projects/search/fred").not_to be_routable
```

Testing Helper Methods

Helper modules are the storage attic of Rails applications. They are designed to contain reusable bits of view logic. This might include view-specific representations of data, or conditional logic that governs how content is displayed. Helper modules tend to get filled with all kinds of clutter that doesn't seem to belong anywhere else. Because they are a little tricky to set up for testing, helper methods often aren't tested even when they contain significant amounts of logic. Again, in my own practice I often move this logic into dedicated presenter classes, but you'll see a lot of code in Rails helpers you might want to unit-test.

RSpec helper tests go in spec/helpers. There's not a whole lot of special magic here—just a helper object that you use to call your helper methods.